euPOWERED: EU Renewable Energy Navigator

Strategic Analytics for Decarbonization, Economic Resilience, and Energy Security (2015–2024)

The comprehensive roadmap for the European energy transition.

It provides lobbyists and policy advocates with the empirical evidence needed to **bridge the gap between climate ambition and economic reality**.

BY DOMINIKA DRĄŻYK
APRIL, 2026

Introduction

This strategic data engine is designed to transform complex Eurostat datasets into actionable political leverage in the **Renewable Energy** discourse.

INTENDED AUDIENCE

- **Lobbyists:** seeking to anchor their renewable energy pitches in validated, cross-border statistics;
- **Policymakers and Advisors:** requiring a high-level overview of national progress relative to EU-wide benchmarks;
- **Publicists and Analysts:** looking to debunk common myths regarding the costs and stability of the green energy transition.

Data source: eurostat

This strategic data engine is designed to transform complex Eurostat datasets into actionable political leverage in the **Renewable Energy** discourse.

INTENDED AUDIENCE

- **Lobbyists:** seeking to anchor their renewable energy pitches in validated, cross-border statistics;
- **Policymakers and Advisors:** requiring a high-level overview of national progress relative to EU-wide benchmarks;
- **Publicists and Analysts:** looking to debunk common myths regarding the costs and stability of the green energy transition.

[ten00124] Final energy consumption by sector

[nrg_ind_ren] Share of energy from renewable sources

[nrg_pc_204] Electricity prices for household consumers (bi-annual)

↓

publicly available

regularly updated

structured, but requires pre-processing

Approach

Phase 1: Data Extraction & Web Scraping

Step 1: Extracting datasets via official API and web scraping

Step 2: Performing the outer merge for the obtained datasets

Language: Python (*BeautifulSoup, selenium, pyjstat)*

Code: *scraper_code.py*

Phase 2: Preprocessing & Data Audit

Step 3: Renaming dimension codes to user-friendly labels

Step 4: Systematic missing-data audit

Language: Python (*pandas, numpy*)

Code: *preproc_code.py*

Phase 3: Visualizations & Dashboard

Step 5: Preparing data visualizations

Step 6: Rendering an ineractive dashboard to present results

Language: Python (*seaborn, matplotlib, Streamlit*)

Code: *figures.py, dashboard.py*

Example – outer merge

```python
def _safe_outer_merge(left: pd.DataFrame, right: pd.DataFrame, label: str) -> pd.DataFrame:
    right2 = right.copy()
    collisions = set(left.columns).intersection(set(right2.columns)) - set(WIDE_INDEX)
    if collisions:
        right2 = right2.rename(columns={
            c: c.replace("_value", f"_{label}_value") for c in collisions})
    return pd.merge(left, right2, on=WIDE_INDEX, how="outer", sort=True)
```

SOLUTION

This function executes a sorted outer merge between two DataFrames while resolving potential naming collisions.

Phase 1: Data Extraction & Web Scraping

Step 1: Extracting datasets via official API and web scraping

Step 2: Performing the outer merge for the obtained datasets

Language: Python (*BeautifulSoup, selenium, pyjstat)*

Code: *scraper_code.py*

Phase 2: Preprocessing & Data Audit

Step 3: Renaming dimension codes to user-friendly labels

Step 4: Systematic missing-data audit

Language: Python (*pandas, numpy*)

Code: *preproc_code.py*

Phase 3: Visualizations & Dashboard

Step 5: Preparing data visualizations

Step 6: Rendering an ineractive dashboard to present results

Language: Python (*seaborn, matplotlib, Streamlit*)

Code: *figures.py, dashboard.py*

Example – missing data audit

```
print("---- O1.3.0-2 The percentage of data entry gaps per country:")
df_nans_ten00124 = df.groupby('geo')[['Consumption Industry',
                                      'Consumption Transport',
                                      'Consumption Households']].apply(
lambda x: (x.isna().sum() * 100 / len(x) )).sort_values(by = 'geo')

df_nans_nrg_ind_ren = df.groupby('geo')[['REN Share',
                                         'REN Share Transport',
                                         'REN Share Heat-Cool',
                                         'REN Share Electricity']].apply(
lambda x: (x.isna().sum() * 100 / len(x) )).sort_values(by = 'geo')

df_nans_nrg_pc_204 = df.groupby('geo')[['Price+Taxes',
                                        'Price']].apply(
lambda x: (x.isna().sum() * 100 / len(x) )).sort_values(by = 'geo')
```

SOLUTION

The script performs a data completeness audit by calculating the percentage of missing values for each energy indicator, grouped and sorted by geographic code across the three source datasets.

Phase 1: Data Extraction & Web Scraping

Step 1: Extracting datasets via official API and web scraping

Step 2: Performing the outer merge for the obtained datasets

Language: Python (*BeautifulSoup, selenium, pyjstat)*

Code: *scraper_code.py*

Phase 2: Preprocessing & Data Audit

Step 3: Renaming dimension codes to user-friendly labels

Step 4: Systematic missing-data audit

Language: Python (*pandas, numpy*)

Code: *preproc_code.py*

Phase 3: Visualizations & Dashboard

Step 5: Preparing data visualizations

Step 6: Rendering an ineractive dashboard to present results

Language: Python (*seaborn, matplotlib, Streamlit*)

Code: *figures.py, dashboard.py*

Example – visualizations

```python
def figure_share_vs_price_correlation(
    df: pd.DataFrame,
    price_col: str = "Price",
    figsize: tuple[float, float] | None = None,
    show_legend: bool = True,
) -> plt.Figure:
    # Figure 4 — Scatterplot correlating Renewable Share [nrg_bal: REN] with consumer
    # energy costs. It features a dynamic y-axis that switches between
    # Price+tax [tax: I_TAX] and Price [tax: X_TAX] based on user selection.
    # Plot clusters data points by color per Country, representing annual
    # observations from 2015 to 2024.
    size = _resolve_figsize(figsize)
    if "REN Share" not in df.columns or price_col not in df.columns:
        return _empty_fig(f"Need REN Share and {price_col}", size)

    plot_df = df.dropna(subset=["REN Share", price_col]).copy()
    if plot_df.empty:
        return _empty_fig("No data", size)

    fig, ax = plt.subplots(figsize=size)
    sns.scatterplot(data=plot_df,x="REN Share",y=price_col,hue="Country",
        palette=FIGURE234_SERIES_PALETTE,ax=ax,alpha=0.85)
    ax.set_xlabel("Total Renewable Energy Share (%)")
    ax.set_ylabel("Household Electricity Price (EUR/kWh)")
    ax.set_title("Historical Correlation: Renewable Penetration vs. Energy Pricing")
    if show_legend:
        ax.legend(title="Country",bbox_to_anchor=(1.01, 1),
                  loc="upper left",borderaxespad=0)
        _apply_legend_fontsizes(ax, 9, 10)
    else:
        ax.legend().remove()
    plt.tight_layout()
    return fig
```

SOLUTION

The script generates a scatterplot correlating Total Renewable Energy Share with consumer energy costs.

Phase 1: Data Extraction & Web Scraping

Step 1: Extracting datasets via official API and web scraping

Step 2: Performing the outer merge for the obtained datasets

Language: Python (*BeautifulSoup, selenium, pyjstat)*

Code: *scraper_code.py*

Phase 2: Preprocessing & Data Audit

Step 3: Renaming dimension codes to user-friendly labels

Step 4: Systematic missing-data audit

Language: Python (*pandas, numpy*)

Code: *preproc_code.py*

Phase 3: Visualizations & Dashboard

Step 5: Preparing data visualizations

Step 6: Rendering an ineractive dashboard to present results

Language: Python (*seaborn, matplotlib, Streamlit*)

Code: *figures.py, dashboard.py*

Example – visualizations

```python
def _page_title_and_subtitle(title: str, subtitle: str) -> None:
    """Centered header; each line +3pt vs typical Streamlit title / h3.
    The substring ``euPOWERED`` in *title* is rendered in the brand accent colour.
    """
    accent = "var(--brand-accent)"
    title_html = html.escape(title).replace(
        html.escape("euPOWERED"),
        f'<span style="color:{accent};">euPOWERED</span>',
    )
    st.markdown(
        f'<div style="text-align:center;">'
        f'<p style="font-size:calc(2.75rem + 3pt);font-weight:700;margin:0 0 0.4rem 0;'
        f'line-height:1.2;">{title_html}</p>'
        f'<p style="font-size:calc(1.35rem + 3pt);font-weight:600;margin:0;line-height:1.35;">'
        f"{html.escape(subtitle)}</p>"
        f"</div>",
        unsafe_allow_html=True,
    )
```

SOLUTION

The script renders a centered, custom-styled header for the dashboad precisely scaling the font sizes of the title and subtitle via injected HTML.

Phase 1: Data Extraction & Web Scraping

Step 1: Extracting datasets via official API and web scraping

Step 2: Performing the outer merge for the obtained datasets

Language: Python (*BeautifulSoup, selenium, pyjstat)*

Code: *scraper_code.py*

Phase 2: Preprocessing & Data Audit

Step 3: Renaming dimension codes to user-friendly labels

Step 4: Systematic missing-data audit

Language: Python (*pandas, numpy*)

Code: *preproc_code.py*

Phase 3: Visualizations & Dashboard

Step 5: Preparing data visualizations

Step 6: Rendering an ineractive dashboard to present results

Language: Python (*seaborn, matplotlib, Streamlit*)

Code: *figures.py, dashboard.py*

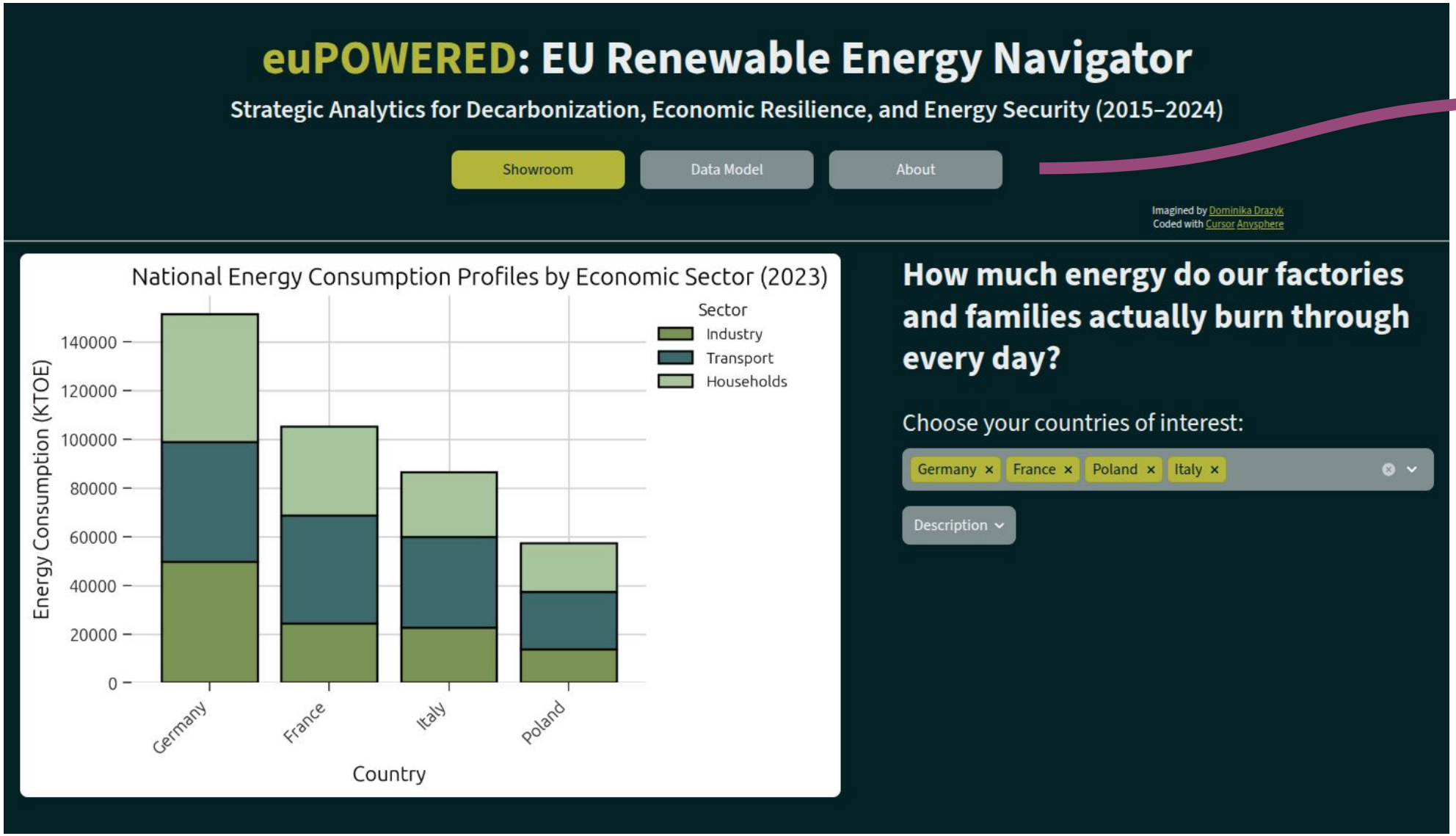
AI-augumented workflow

The project was built through an **AI-augmented workflow**, using the Cursor IDE and Claude.

Data extraction, preprocessing, and analytical figure logic were written by the Author and optimized by Cursor. Presentation layer (Streamlit, CSS branding, HTML rendering, code refactoring) were implemented by Cursor under the following rules:

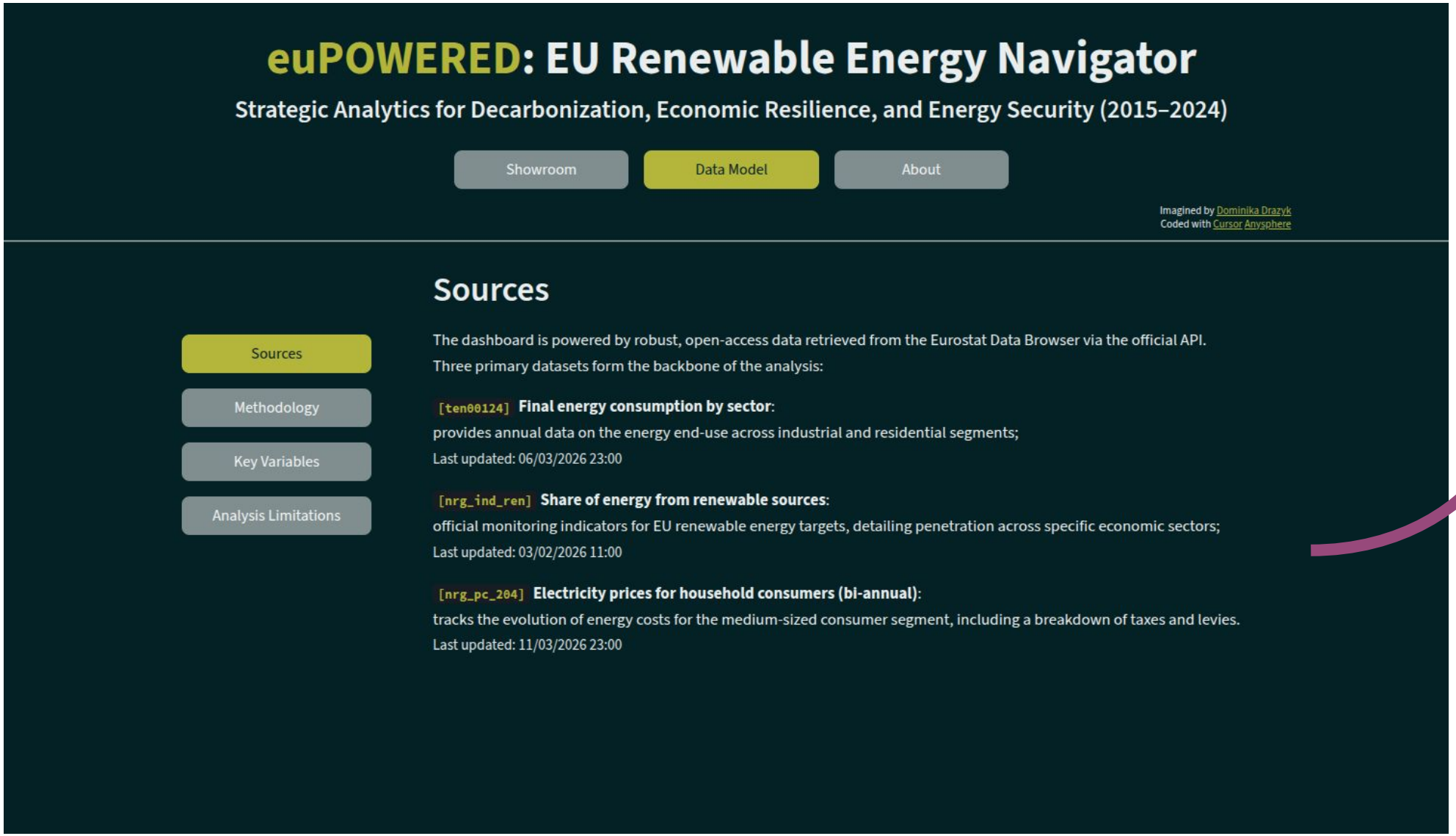
- **Author-driven design decisions**: All visual design choices (e.g, color palette, layout proportions, typography), content, and analytics (e.g, how to group RES Shares) originated from the Author;
- **AI-assisted implementation**: Cursor translated high-level instructions into working code, handling the CSS specificity with Streamlit's internal styles, HTML templating, and Matplotlib API details;
- **Continuous review**: The Author verified every change and provided feedback. The Author also made direct edits to the Streamlit code independently between Cursor sessions. Large structural improvements were first proposed by Cursor, then selectively approved or rejected by the Author before implementation.

The Dashboard



Navigation Buttons to switch between dashboard pages.

The Dashboard



The Data Model page that synchronizes after dataset update including Eurostat [Links] and metadata (*Last updated*).

The Dashboard

euPOWERED: EU Renewable Energy Navigator

Strategic Analytics for Decarbonization, Economic Resilience, and Energy Security (2015–2024)

Showroom | Data Model | About

Imagined by Dominika Drazyk
Coded with Cursor Anysphere

About

EU-powered is a strategic data engine designed to transform complex Eurostat datasets into actionable political leverage. This comprehensive roadmap for the European energy transition, provides lobbyists and policy advocates with the empirical evidence needed to **bridge the gap between climate ambition and economic reality**.

Intended Audience

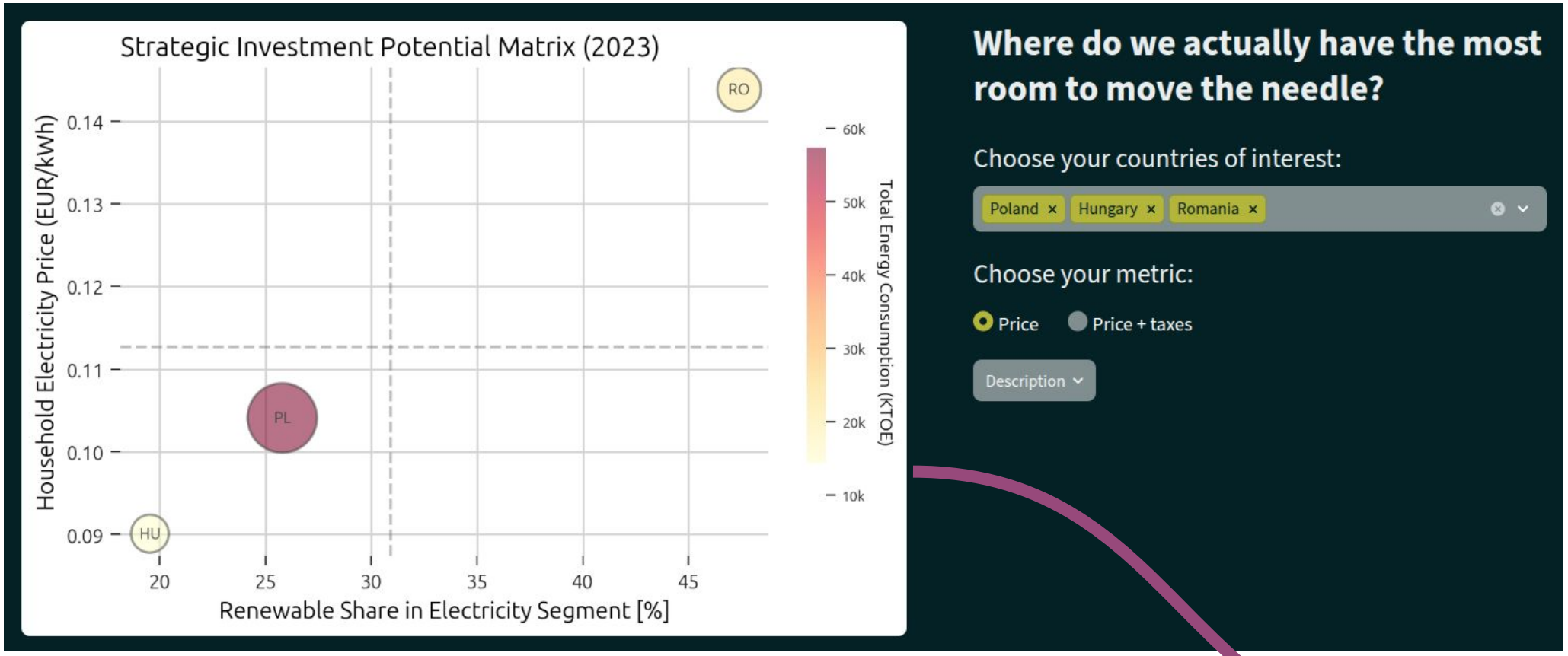
- **Lobbyists**: seeking to anchor their renewable energy pitches in validated, cross-border statistics;
- **Policymakers and Advisors**: requiring a high-level overview of national progress relative to EU-wide benchmarks;
- **Publicists and Analysts**: looking to debunk common myths regarding the costs and stability of the green energy transition.

Strategic Applications

- **Present the affordability argument**, by correlating high renewable penetration with long-term price stability and lower net costs;
- **Expose sectoral bottlenecks** by identifying exactly where heating and transport systems are failing to keep pace with the power grid;
- **Optimize capital flow** by pinpointing the "High Potential" markets where infrastructure investment will deliver the most significant economic and environmental ROI;
- **Test national decarbonization performance**, by direct, evidence-based comparisons between member states.

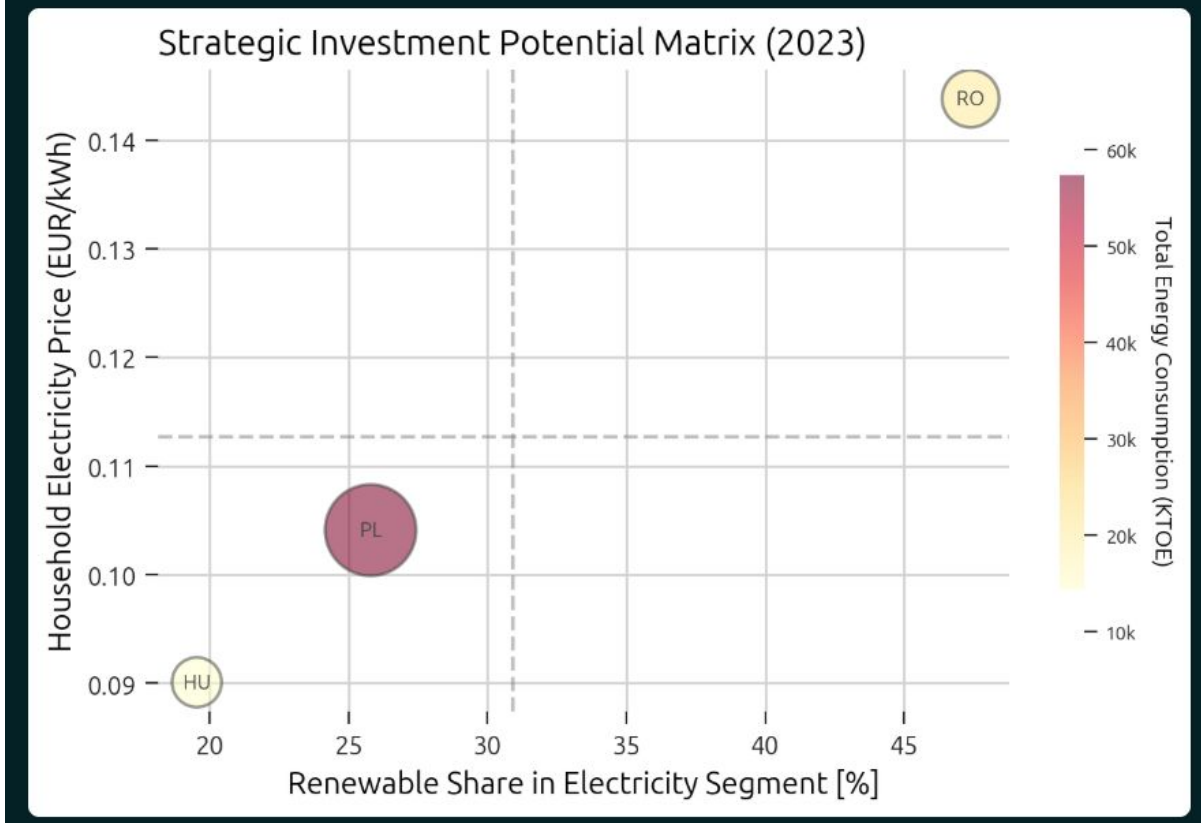
The About page including basic project information and data-driven strategic insights.

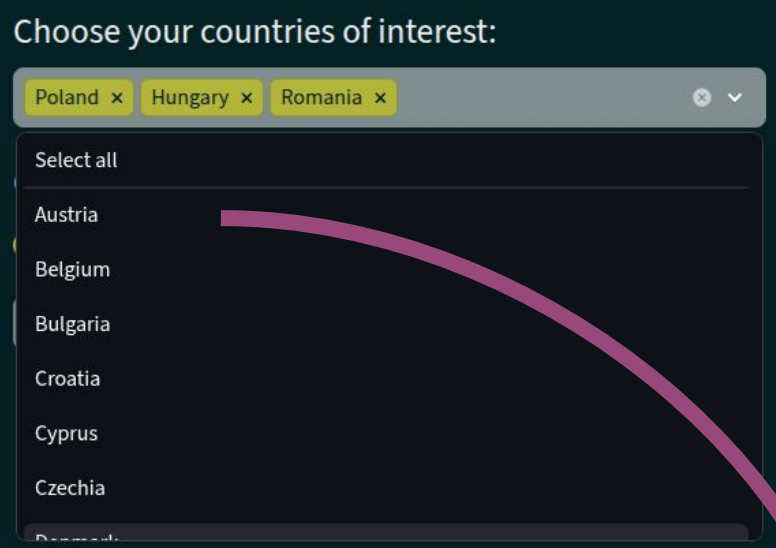
The Dashboard



In the Showroom page, each data visualization changes in real time, depending on chosen parameters...

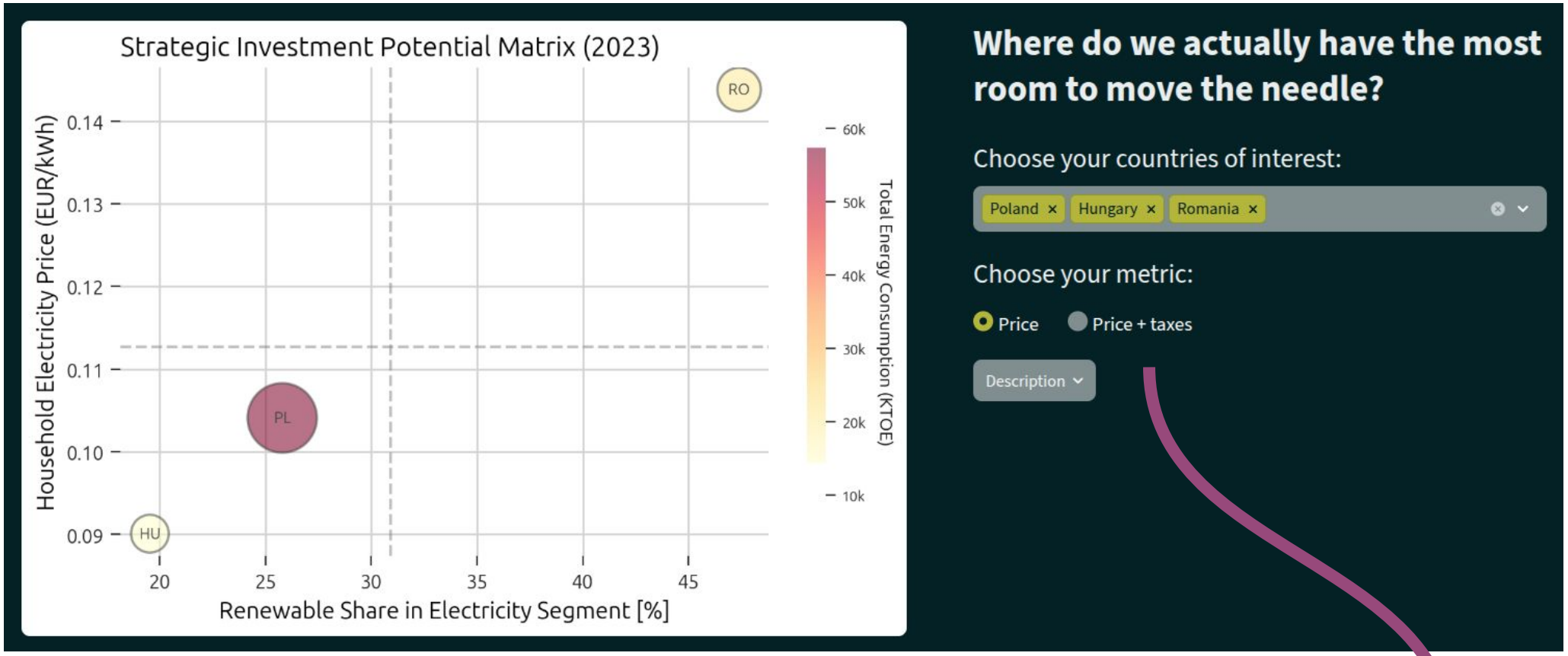
The Dashboard





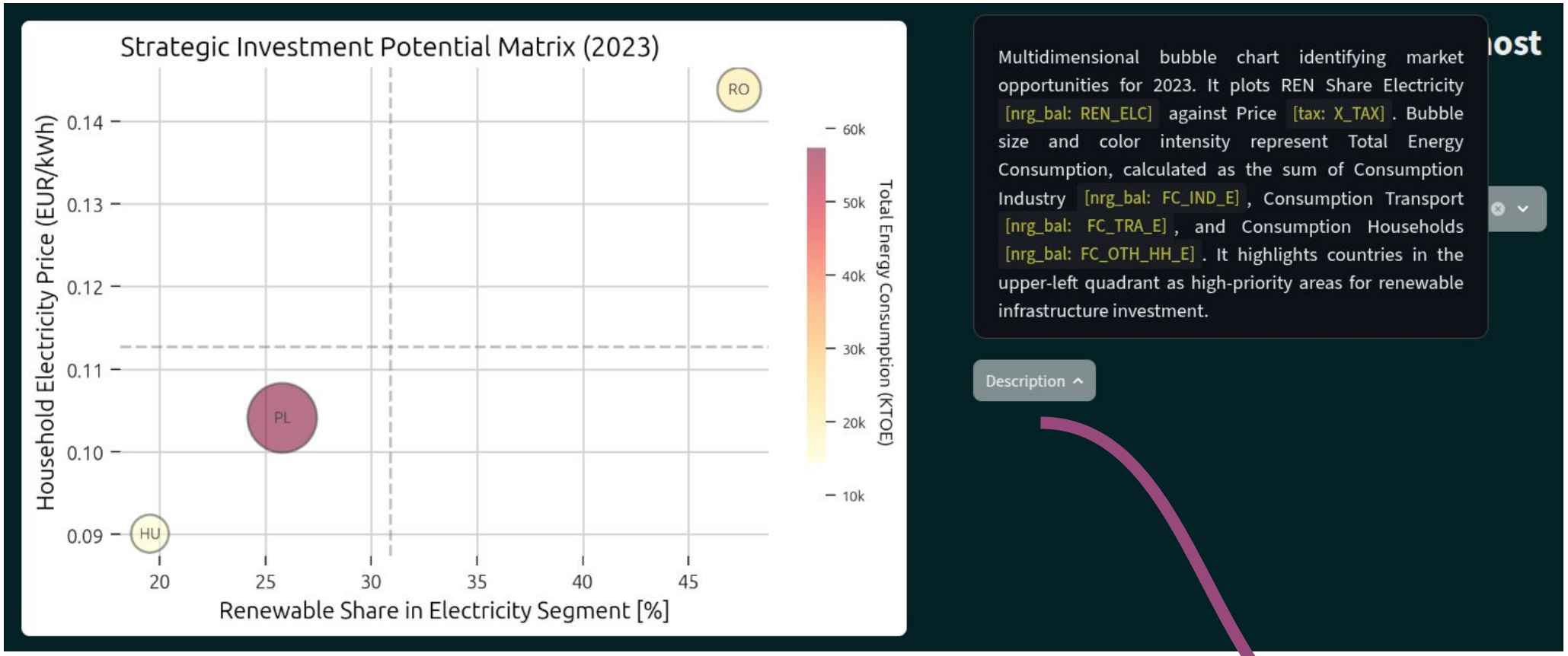
… Countries of interest, …

The Dashboard



… or chosen Price calculation.

The Dashboard



Description buttons provide technical information and data sources for each visualization.

Practical business aplications

- **Present the affordability argument,** by correlating high renewable penetration with long-term price stability and lower net costs;
- **Expose sectoral bottlenecks** by identifying exactly where heating and transport systems fail to keep pace with the power grid;
- **Optimize capital flow** by pinpointing the “High Potential” markets where infrastructure investment will deliver the most significant economic and environmental ROI;
- **Test national decarbonization performance**, by direct, evidence-based comparisons between member states.

- **Present the affordability argument,** by correlating high renewable penetration with long-term price stability and lower net costs;
- **Expose sectoral bottlenecks** by identifying exactly where heating and transport systems fail to keep pace with the power grid;
- **Optimize capital flow** by pinpointing the "High Potential" markets where infrastructure investment will deliver the most significant economic and environmental ROI;
- **Test national decarbonization performance**, by direct, evidence-based comparisons between member states.

Analysis limitations

- *By design, the pipeline omits the Commercial and Public Services. Total figures represent Industry, Transport, and Households sectors (covering the vast majority of consumption), not the absolute national total;*
- *Pricing data is anchored to the DC Band from 2500 to 4999 kWh (median EU household). It may not reflect the costs faced by low-income households or heavy industrial users;*
- *An "Outer Merge" strategy is used to accommodate countries with incomplete records for specific years. Consequently, some figures may appear fragmented for specific Member States.*

Important note:
The insights provided are focused on the strategic application of energy metrics for policy advocacy and are not based on formal academic expertise in energy economics or legislative drafting.

My aim was to demonstrate the ability to architect data, design robust ETL processes, and extract actionable business intelligence from complex relational datasets.

Results

{ scraper_data.csv }
{ scraper_metadata.csv }
{ eu_efta_countries.csv }

Eurostat-related datasets, extracted via API or scraped;

{ preprocessed_data.csv }

main preprocessed dataset ready for data anlaysis;

scraper_code.py

Python data extraction code;

preproc_code.py

Python preprocessing code;

figures.py

Python code for figure generation;

dashboard.py

Python dashboard rendering code;

brand.css

stylesheet for the dashboard;

Key skills

Data modelling & engineering

- Developing **custom web scrapers** to automate the extraction of public datasets and metadata.
- Preparing and **cleaning datasets** (synchronizing semi-annual series, mapping technical codes to human-readable labels).
- Executing systematic **missing-data audits** and diagnostic visualizations to maintain the structural integrity of the unified data model.
- Leveraging **AI-augmented development** while maintaining strict authorial oversight of analytical logic and visual proportions.

Data visualization & storytelling

- **Imagining an interactive dashboard** with a custom-branded CSS design system for reporting.
- Translating abstract statistics into a **strategic lobbyist narrative** focused on affordability and decarbonization progress.
- Utilizing **advanced visual analytics**, to identify high-yield investment opportunities and sectoral bottlenecks.
- Preparing comprehensive documentation to effectively **communicate methodology and limitations**.